CSTBS / RELEASE REPORT

Version 2.0
Accounts & Events

Campus Stadium Ticket Booking System | Release date: November 1, 2026

V2.0 turns the original read-only prototype into an account-enabled event platform with real sessions, secure password storage, role-based access and administrator event management.

What changed from Version 1

Area	Version 1	Version 2.0
Accounts	UI-only login and registration	Real student registration, login and POST logout
Security	Prototype forms with CSRF	Flask-Login sessions, Werkzeug password hashes, role checks
Events	Public read-only listing	Same public catalog plus admin create, edit and delete
Sample data	Automatically seeded on startup	Explicit, repeatable CLI seed command
SQLite schema	User table had no role	User.role with student/staff/admin and V1 upgrade
Project delivery	README and design docs	Updated docs, changelog, testing and release guide

Scope boundary

This release does not include bookings, seat selection, ticket inventory, payments, password recovery, email verification, rate limiting, or production deployment. Seat and Booking remain schema placeholders.

FEATURES & ARCHITECTURE

Implementation details

New capabilities

• User accounts: registration writes users to SQLite; passwords use Werkzeug hashing; login and logout use Flask-Login.

• Role controls: student, staff and admin roles; all browser registrations receive student; admin assignment is by trusted CLI only.

• Route protection: account and admin URLs require authentication; unauthorized staff/students receive HTTP 403 for admin pages.

• Admin CRUD: create, edit and delete events through Flask-WTF validated, CSRF-protected forms. Delete requires POST and is blocked for events with seats.

• V1 compatibility: SQLite legacy users table receives a default student role without dropping records; all other tables remain intact.

• Event seeds: one explicit command inserts four sample events only when the catalog is empty; server restarts do not restore removed events.

Notable files added

File	Purpose
CHANGELOG.md	Version 2.0 feature and security history
app/migrations.py	Narrow SQLite V1 user-role schema upgrade
app/templates/account.html	Authenticated account dashboard
app/templates/admin/events.html	Admin event management index
app/templates/admin/event_form.html	Reusable admin event create/edit form
tests/test_app.py	Flask integration tests for auth, permissions, CRUD, CSRF and seed

Core files updated

app/__init__.py, app/models.py, app/routes.py, app/forms.py, app/seed.py, config.py, run.py, requirements.txt, all account/navigation templates, responsive CSS/JS, README and all three docs/*.md design documents.

RELEASE CHECKLIST

Set up & publish v2.0

Quick local start

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
cp .env.example .env
python -m flask --app run.py seed-events
python run.py
```

Generate a new random SECRET_KEY and replace the .env placeholder before running. On Windows, activate with .venv\Scripts\Activate.ps1 and copy the example with Copy-Item.

Administrator bootstrap

Register a normal account first, then locally run:

```
python -m flask --app run.py set-role you@campus.edu admin
```

Sign out and back in to refresh the account view, then visit /admin/events. For V1 users, back up instance/cstbs.db and copy it into the V2 instance directory before the first startup.

Testing and verification

Delivered: Python syntax checks, Jinja2 template compilation, and SVG parsing passed. Flask integration test code is included, but could not be executed in the build runner because Flask dependencies are not installed and network downloads failed. This is not a production security certification.

```
python -m unittest discover -s tests -v
```

Suggested GitHub release metadata

Field	Suggested value
Tag	v2.0
Release title	CSTBS v2.0 - Accounts and Events
Commit	feat: add CSTBS v2.0 accounts and admin event management
Summary	Working account registration/login, roles and protected routes, admin event CRUD, sample seeding, V1 SQLite compatibility, docs and tests.

Before tagging: install dependencies locally, run tests, verify login and admin event operations, and confirm your database backup. Keep .env and instance/*.db out of Git.